

stroke-linecap="round" stroke-linejoin="round"></path> <path d="M7.57393 8.20877C8.51036 8.20877 9.26949 7.44964 9.26949 6.51321C9.26949 5.57678 8.51036 4.81766 7.57393 4.81766C6.6375 4.81766 5.87837 5.57678 5.87837 6.51321C5.87837 7.44964 6.6375 8.20877 7.57393 8.20877Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M7.59297 13.178C10.8019 13.178 13.4033 10.5766 13.4033 7.36767C13.4033 4.1587 10.8019 1.55731 7.59297 1.55731C4.384 1.55731 1.78261 4.1587 1.78261 7.36767C1.78261 10.5766 4.384 13.178 7.59297 13.178Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M30.2116 30.7659C29.1532 29.8267 27.7872 29.308 26.3722 29.308C24.9571 29.308 23.5912 29.8267 22.5327 30.7659" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M26.3743 27.3036C27.3107 27.3036 28.0698 26.5445 28.0698 25.6081C28.0698 24.6716 27.3107 23.9125 26.3743 23.9125C25.4378 23.9125 24.6787 24.6716 24.6787 25.6081C24.6787 26.5445 25.4378 27.3036 26.3743 27.3036Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M26.3943 32.2729C29.6032 32.2729 32.2046 29.6715 32.2046 26.4625C32.2046 23.2535 29.6032 20.6521 26.3943 20.6521C23.1853 20.6521 20.5839 23.2535 20.5839 26.4625C20.5839 29.6715 23.1853 32.2729 26.3943 32.2729Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> </g> <defs> <clipPath id="clip0760814931"> <rect width="31.9954" height="31.9954" fill="white" transform="translate(0.995361 0.917358)"></rect> </clipPath> </defs> </svg>

<p class="h3 my-3">Remote-work essentials</p>

<p class="h5 my-4">Compare: All-Remote vs. Hybrid-Remote <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">How to transition a company to remote operation <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">What not to do when implementing remote work <i class="fa-solid fa-chevron-right"></i></p>

</div>

<div class="col p-4 m-2 bg-light rounded-3" style="min-width: 300px;">

<svg width="33" height="33" viewBox="0 0 33 33" fill="currentColor" fill-opacity="0" xmlns="https://www.w3.org/2000/svg"> <g clip-path="url(clip0760814935)"> <path d="M15.4182 5.72589L17.1876 7.91758C17.8479 8.7318 18.2073 9.74883 18.205 10.7972V11.6098C18.205 12.3736 17.9017 13.106 17.3616 13.646C16.8216 14.186 16.0892 14.4894 15.3255 14.4894H13.7577C13.4689 14.4882 13.1827 14.5439 12.9155 14.6534C12.6483 14.763 12.4054 14.9242 12.2006 15.1278C11.9958 15.3314 11.8332 15.5734 11.7221 15.8399C11.611 16.1065 11.5536 16.3923 11.5532 16.6811V17.321C11.554 17.9026 11.7857 18.46 12.1972 18.871C12.6087 19.2819 13.1665 19.5127 13.7481 19.5127H13.8025C14.2966 19.5127 14.7706 19.7088 15.1203 20.0579C15.4701 20.407 15.667 20.8807 15.6678 21.3748V21.4324C15.6678 21.9416 15.4655 22.4299 15.1055 22.7899C14.7455 23.1499 14.2572 23.3522 13.7481 23.3522H7.66896" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M26.6267 11.002L22.4321 13.3312C22.0468 13.5392 21.7246 13.8472 21.4994 14.2227C21.2742 14.5982 21.1543 15.0274 21.1523 15.4653V17.4874C21.1523 17.8803 21.3084 18.2571 21.5862 18.5349C21.864 18.8127 22.2408 18.9688 22.6337 18.9688V18.9688C22.9582 18.968 23.278 19.0466 23.5652 19.1978C23.8524 19.3489 24.0983 19.568 24.2814 19.8359" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M18.7328 28.1226C18.1578 28.2101 17.5771 28.254 16.9955 28.2538C15.2211 28.2538 13.4717 27.8364 11.8885 27.0353C10.3054 26.2342 8.93287 25.0719 7.88198 23.6422C6.8311 22.2126 6.1313 20.5558 5.83915 18.8057C5.54699 17.0556 5.67068 15.2613 6.20022 13.5679" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M15.4202 5.72587C15.9418 5.65456 16.4679 5.62035 16.9943 5.62349C18.7908 5.62449 20.5613 6.05316 22.1592 6.87406C23.7572 7.69495 25.1368 8.88449 26.1838

10.3443C27.2309 11.8041 27.9154 13.4922 28.1807 15.2689C28.4461 17.0457 28.2846 18.8601 27.7096 20.5621" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M22.5489 32.2821H31.0468C31.3894 32.2846 31.7191 32.1514 31.9637 31.9116C32.2083 31.6718 32.348 31.3448 32.3522 31.0022V25.3551C32.3472 25.0139 32.2081 24.6884 31.965 24.4489C31.722 24.2094 31.3944 24.0752 31.0532 24.0752H27.7513L25.3133 22.0179V24.0593H22.5489C22.2107 24.0634 21.8876 24.1996 21.6485 24.4387C21.4094 24.6778 21.273 25.0009 21.269 25.3391V31.0022C21.269 31.3417 21.4039 31.6672 21.6439 31.9072C21.8839 32.1472 22.2094 32.2821 22.5489 32.2821Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M11.4389 1.56647H2.94092C2.76995 1.5652 2.60042 1.59779 2.44209 1.66234C2.28377 1.72689 2.13978 1.82214 2.01844 1.9426C1.89709 2.06305 1.80078 2.20633 1.73506 2.36417C1.66934 2.52201 1.63551 2.6913 1.63551 2.86228V8.49347C1.63593 8.66433 1.67005 8.83343 1.73592 8.99108C1.80179 9.14873 1.89812 9.29184 2.01938 9.41221C2.14064 9.53258 2.28445 9.62784 2.44259 9.69255C2.60072 9.75725 2.77007 9.79013 2.94092 9.78928H6.23645L8.6745 11.8306V9.78928H11.4517C11.7926 9.78507 12.1181 9.6467 12.3576 9.40414C12.5972 9.16158 12.7315 8.83439 12.7315 8.49347V2.86228C12.7337 2.69179 12.7017 2.52259 12.6375 2.36462C12.5734 2.20665 12.4783 2.06309 12.3579 1.94237C12.2375 1.82166 12.0941 1.72622 11.9363 1.66167C11.7785 1.59712 11.6094 1.56475 11.4389 1.56647V1.56647Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> </g> <defs> <clipPath id="clip0760814935"> <rect width="31.9954" height="31.9954" fill="white" transform="translate(0.995361 0.926514)"></rect> </clipPath> </defs> </svg> <p class="h3 my-3">Designing a remote team</p> <p class="h5 my-4">About the Head of Remote role <i class="fa-solid fa-chevron-right"></i></p> <p class="h5 my-4">The 10 models of remote teams <i class="fa-solid fa-chevron-right"></i></p> <p class="h5 my-4">The importance of documentation for remote teams <i class="fa-solid fa-chevron-right"></i></p> </div> </div>

<div class="row" style="max-width: 80%; margin-left: auto; margin-right: auto;"> <div class="col p-4 m-2 bg-light rounded-3" style="min-width: 300px;"> <svg width="33" height="33" viewBox="0 0 33 33" fill="currentColor" fill-opacity="0" xmlns="https://www.w3.org/2000/svg"> <g clip-path="url(clip0760814923)"> <path d="M4.351 24.9684V9.93055C4.351 9.52324 4.5128 9.13261 4.80082 8.84459C5.08883 8.55658 5.47946 8.39478 5.88678 8.39478H10.8781" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M29.6352 24.9684V9.93055C29.6352 9.52324 29.4733 9.13261 29.1853 8.84459C28.8973 8.55658 28.5067 8.39478 28.0994 8.39478H23.2521" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M3.17015 28.6223C2.76283 28.6223 2.3722 28.4605 2.08419 28.1724C1.79617 27.8844 1.63437 27.4938 1.63437 27.0865V24.9684H32.3499V27.0865C32.3499 27.4938 32.1881 27.8844 31.9001 28.1724C31.6121 28.4605 31.2215 28.6223 30.8142 28.6223H3.17015Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M24.3889 20.5786C24.3889 12.7941 10.1989 12.7685 10.1989 20.5786H24.3889Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M17.2964 12.0966C19.1818 12.0966 20.7103 10.5682 20.7103 8.68271C20.7103 6.79726 19.1818 5.2688 17.2964 5.2688C15.4109 5.2688 13.8824 6.79726 13.8824 8.68271C13.8824 10.5682 15.4109 12.0966 17.2964 12.0966Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> </g> <defs> <clipPath id="clip0760814923"> <rect width="31.9954" height="31.9954" fill="white"

transform="translate(0.995361 0.949402)"></rect> </clipPath> </defs> </svg>

<p class="h3 my-3">Remote hiring & onboarding</p>

<p class="h5 my-4">How to hire for a remote team <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">The guide to remote onboarding <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">How to evaluate a remote job opportunity <i class="fa-solid fa-chevron-right"></i></p>

</div>

<div class="col p-4 m-2 bg-light rounded-3" style="min-width: 300px;">

<svg width="32" height="33" viewBox="0 0 32 33" fill="currentColor" fill-opacity="0"

xmlns="https://www.w3.org/2000/svg"> <g clip-path="url(clip0760814965)"> <path d="M4.97144

17.0143C5.80529 19.2113 7.31272 21.0883 9.27801 22.3767C9.56103 22.5663 9.7927 22.823 9.95237

23.1239C10.112 23.4248 10.1947 23.7606 10.1931 24.1013V27.144C10.1914 28.521 10.7363 29.8423

11.7082 30.8178C12.68 31.7933 13.9994 32.3431 15.3763 32.3465V32.3465C16.0595 32.3469 16.736

32.2126 17.3672 31.9512C17.9984 31.6899 18.5719 31.3067 19.0548 30.8234C19.5377 30.3402 19.9206

29.7665 20.1816 29.1352C20.4425 28.5038 20.5764 27.8272 20.5756 27.144V24.1013C20.5744 23.6371

20.73 23.1861 21.0171 22.8214" stroke="currentColor" stroke-width="1.27982" stroke-

linecap="round" stroke-linejoin="round"></path> <path d="M24.7044 7.04123C23.6472 5.41078

22.1807 4.08619 20.4515 3.19968C18.7223 2.31318 16.7908 1.89578 14.8498 1.98915C13.8133 2.03543

12.7888 2.22936 11.8071 2.56507" stroke="currentColor" stroke-width="1.27982" stroke-

linecap="round" stroke-linejoin="round"></path> <path d="M10.1765 24.1012H20.575"

stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-

linejoin="round"></path> <path d="M9.39221 26.8496H21.3585" stroke="currentColor" stroke-

width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M10.0903

29.5979H20.8375" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-

linejoin="round"></path> <path d="M11.6615 13.2099C10.4499 12.1377 8.88795 11.5458 7.27009

11.5458C5.65224 11.5458 4.09028 12.1377 2.87872 13.2099" stroke="currentColor" stroke-

width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M7.26612

9.2461C8.33708 9.2461 9.20527 8.37792 9.20527 7.30695C9.20527 6.23599 8.33708 5.3678 7.26612

5.3678C6.19515 5.3678 5.32697 6.23599 5.32697 7.30695C5.32697 8.37792 6.19515 9.2461 7.26612

9.2461Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-

linejoin="round"></path> <path d="M7.28982 14.9312C10.9618 14.9312 13.9385 11.9545 13.9385

8.28255C13.9385 4.61061 10.9618 1.63391 7.28982 1.63391C3.61787 1.63391 0.641174 4.61061

0.641174 8.28255C0.641174 11.9545 3.61787 14.9312 7.28982 14.9312Z" stroke="currentColor"

stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M29.076

19.2284C27.8637 18.1526 26.2991 17.5586 24.6783 17.5586C23.0575 17.5586 21.4929 18.1526 20.2805

19.2284" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-

linejoin="round"></path> <path d="M24.6801 15.2069C25.751 15.2069 26.6192 14.3387 26.6192

13.2678C26.6192 12.1968 25.751 11.3286 24.6801 11.3286C23.6091 11.3286 22.7409 12.1968 22.7409

13.2678C22.7409 14.3387 23.6091 15.2069 24.6801 15.2069Z" stroke="currentColor" stroke-

width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path d="M24.7013

20.9496C28.3733 20.9496 31.35 17.9729 31.35 14.301C31.35 10.629 28.3733 7.65234 24.7013

7.65234C21.0294 7.65234 18.0527 10.629 18.0527 14.301C18.0527 17.9729 21.0294 20.9496 24.7013

20.9496Z" stroke="currentColor" stroke-width="1.27982" stroke-linecap="round" stroke-

linejoin="round"></path> </g> <defs> <clipPath id="clip0760814965"> <rect width="31.9954"

height="31.9954" fill="white" transform="translate(0 0.990845)"></rect> </clipPath> </defs>

</svg>

<p class="h3 my-3">Collaboration & async</p>

<p class="h5 my-4">How to work asynchronously <i class="fa-

solid fa-chevron-right"></i></p>

<p class="h5 my-4">How to run effective meetings remotely <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">Remote collaboration and whiteboarding <i class="fa-solid fa-chevron-right"></i></p>

</div>

</div>

<div class="row" style="max-width: 80%; margin-left: auto; margin-right: auto;">

<div class="col p-4 m-2 bg-light rounded-3" style="min-width: 300px;">

<svg width="33" height="32" viewBox="0 0 32 32" fill="currentColor"

xmlns="https://www.w3.org/2000/svg"> <g clip-path="url(clip01083613664)"> <path d="M23.9 32H0.0799561L3.41996 25.72C4.63996 23.44 6.99996 22 9.59996 22H14.38C16.98 22 19.36 23.42 20.56 25.72L23.88 32H23.9ZM3.39996 30H20.58L18.82 26.66C17.96 25.02 16.26 24 14.4 24H9.61996C7.75996 24 6.07996 25.02 5.19996 26.66L3.41996 30H3.39996Z"></path> <path d="M12 20C8.7 20 6 17.3 6 14C6 10.7 8.7 8 12 8C15.3 8 18 10.7 18 14C18 17.3 15.3 20 12 20ZM12 10C9.8 10 8 11.8 8 14C8 16.2 9.8 18 12 18C14.2 18 16 16.2 16 14C16 11.8 14.2 10 12 10Z"></path> <path d="M32.1599 26H23.5199V24H28.2999L26.9799 22.12C26.0399 20.8 24.5199 20 22.8999 20H20.0199V18H22.8999C25.1799 18 27.3199 19.1 28.6199 20.96L32.1599 26Z"></path> <path d="M22.0599 15.88L21.6399 13.92C23.0199 13.62 23.9999 12.4 23.9999 10.98C23.9999 9.31998 22.6599 7.97998 20.9999 7.97998C20.3599 7.97998 19.7599 8.17998 19.2399 8.53998L18.0599 6.91998C18.9199 6.29998 19.9199 5.97998 20.9799 5.97998C23.7399 5.97998 25.9799 8.21998 25.9799 10.98C25.9799 13.32 24.3199 15.38 22.0399 15.86L22.0599 15.88Z"></path> </g> <defs> <clipPath id="clip01083613664"> <rect width="32" height="32" fill="FFFFFF"></rect> </clipPath> </defs> </svg>

<p class="h3 my-3">Scaling remote culture</p>

<p class="h5 my-4">How to build culture on remote teams <i class="fa-solid fa-chevron-right"></i></p>

<p class="h5 my-4">The importance of informal communication <i class="fa-solid fa-chevron-right"></i></p>

</div>

<div class="col p-4 m-2 bg-light rounded-3" style="min-width: 300px;">

<svg width="33" height="32" viewBox="0 0 33 32" fill="none"

xmlns="https://www.w3.org/2000/svg"> <g id="handbook-gitlab"> <path id="Vector" d="M5.00882 3.37281C5.00967 2.64774 5.2983 1.95265 5.81131 1.44024C6.32431 0.927831 7.01974 0.640014 7.74482 0.640015H28.1832V21.9072" stroke="currentColor" fill="none" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path id="Vector2" d="M5.00882 27.2032V6.42883" stroke="currentColor" stroke-width="1.27982" fill="none" stroke-linecap="round" stroke-linejoin="round"></path> <path id="Vector3" d="M25.7288 29.9361H28.1832V24.4705H7.74482C7.38594 24.47 7.0305 24.5403 6.69878 24.6773C6.36706 24.8142 6.06556 25.0151 5.8115 25.2686C5.55744 25.5221 5.35579 25.8231 5.21807 26.1545C5.08034 26.4859 5.00924 26.8412 5.00882 27.2001V27.2001C5.00882 27.5592 5.07961 27.9148 5.21715 28.2466C5.35468 28.5784 5.55627 28.8798 5.81037 29.1336C6.06447 29.3874 6.36611 29.5886 6.69804 29.7258C7.02997 29.8629 7.38567 29.9333 7.74482 29.9329H19.8856" stroke="currentColor" fill="none" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path id="Vector4" d="M25.5238 31.3601L22.8006 30.0225L20.0774 31.3601V26.8833H25.5238V31.3601Z" stroke="currentColor" fill="none" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> <path id="Vector5" d="M16.5993 16.8L21.3289 13.3088C21.3929 13.2615 21.4404 13.1953 21.4646 13.1194C21.4887 13.0435 21.4883 12.962 21.4633 12.8864L19.8345 7.7888C19.8229 7.7507 19.7994 7.71733 19.7674 7.69362C19.7354 7.66991 19.6967 7.6571 19.6569

7.6571C19.6171 7.6571 19.5783 7.66991 19.5463 7.69362C19.5143 7.71733 19.4908 7.7507 19.4793 7.7888L18.3977 11.1744H14.7977L13.7129 7.7888C13.7013 7.7507 13.6778 7.71733 13.6458 7.69362C13.6138 7.66991 13.5751 7.6571 13.5353 7.6571C13.4955 7.6571 13.4567 7.66991 13.4247 7.69362C13.3927 7.71733 13.3692 7.7507 13.3577 7.7888L11.7289 12.8864C11.7039 12.962 11.7034 13.0435 11.7276 13.1194C11.7517 13.1953 11.7992 13.2615 11.8633 13.3088L16.5993 16.8Z" stroke="currentColor" fill="none" stroke-width="1.27982" stroke-linecap="round" stroke-linejoin="round"></path> </g> </svg>

<p class="h3 my-3">Learn more</p>

<p class="h4 my-4">For in-depth resources on everything you need to know about remote work, see GitLab's full Guide to Remote.</p>

<p class="h5 my-4">Read GitLab's full Guide to Remote <i class="fa-solid fa-chevron-right"></i></p>

</div>

</div>

{{< include "includes/take-gitlab-for-a-spin.md" >}}

SECTION: company/culture/all-remote/all-remote-vs-hybrid-remote-comparison.md

{style="max-width: 50%" }

Not all forms of remote work are created equal. While hybrid may appear to offer the best of both worlds on the surface, many leaders underestimate its complexities and embedded inequality. On this page, we'll compare and contrast benefits available to all-remote leaders and team members with those available in a hybrid-remote setting.

This page is designed to clarify the differences in workplace experience by individuals who are remote by default in an all-remote setting vs. a hybrid-remote setting. This page is not designed to compare the in-person experience with the remote experience within a hybrid-remote organization.

It is becoming more common for job seekers to weigh multiple competing offers. Understanding the nuanced differences in operating within a single-tier environment (all-remote) vs. a multi-tier environment (hybrid-remote) is critical for expectation alignment. Be careful to not assume that all remote offers are equal, thereby simplifying job selection to metrics such as compensation and job title. The job selection process is intensely personal, and this guide intends to help those in the process put a more meaningful value on all-remote benefits (and conversely, arm candidates with data if negotiating a higher salary to compensate for the inbuilt dysfunction and inequity present in many hybrid environments).

Language and terminology

For the purpose of this compare and contrast exercise, we define hybrid-remote as an environment where optionality and flexibility is limited. The moment you force a subset of people back into the office, with another subset that is always remote, you have two playing fields to administrate. This two-tier arrangement is extremely difficult to administrate and is unequal by design.

Those who are never remote will default to working office-first (e.g. using a physical whiteboard that cannot support remote contribution) and those who are never in the office will miss out on non-transparent work conversations and promotion opportunities.

For deeper dives on this topic, read our guides below.

1. How to evaluate a remote job

1. The stages of remote work

1. Hybrid-Remote: understanding nuances and pitfalls

1. Why GitLab uses the term all-remote to describe its 100% remote workforce

All-Remote vs. Hybrid-Remote benefits checklist

Benefit	Hybrid-Remote	All-Remote
No commute	Yes	Yes
Non-linear workday	Yes, for roles which allow	No; fixed to office schedule of colleagues
Ability to relocate for personal reasons	Limited	Yes
Ability to optimize location for something other than a commutable distance to an office	Yes	Limited
Avoid physiological stress of open-plan offices	Depends	Yes
Equal footing by default on access to executives and senior leaders	No	Yes
Praise and promotion based on work output vs. "face time" with the right people	Yes	No
IT/support applied to everyone equally	No	Yes
All team members use digital tools to work, collaborate, and communicate	No	Yes
Freedom from manually chasing information shared during siloed, in-person meetings	Yes	No
Freedom from "FOMO" - the fear of missing out on in-person perks, meetings, social gatherings, etc.	Yes	No
Travel burden for in-person culture and rapport building is on the company, not the individual	Yes	Depends
Freedom from managing guilt (e.g. you don't commute; your colleagues do)	No	Yes
Freedom from justifying one's remote work arrangement to managers who aren't naturally supportive of flexibility	Yes	No
Freedom from demands for overperformance by in-office colleagues	No	Yes
Freedom from concerns of being overlooked, forgotten, or dehumanized by in-office managers	Yes	No

| Freedom (and likely reimbursement) to work in a "third space" (e.g. coworking center, Codi, coffee shop, etc.) | Yes | Depends |

| Freedom from subpar hybrid video calls where select participants are huddled in a physical room on one camera | Yes | No |

| Equal footing in job interviews (e.g. no advantage/disadvantage from in-person interviews) | Yes | No |

| Learning and development is applicable to all team members, regardless of location | Yes | No |

| Career growth is not dependent on willingness/ability to transfer to in-person by default | Yes | No |

| Freedom from last-minute demands to fly to an office for an in-person meeting | Yes | No |

| Business operations team is fully focused on upleveling tools and technology which positively impact all team members (vs. catering to one subset over another) | Yes | No |

| Freedom from being transferred to a manager or department who is in-person and/or isn't supportive of remote arrangements | Yes | No |

| Eclectic mix of company-wide in-person gatherings (vs. congregating at a standard office) | Yes | No |

| Hearing about water cooler chitchat and perpetual cubical drop-ins from your in-office colleagues | No | Yes |

Learn more through [Life at GitLab](#) and our [Out of the Office](#) report. If this page resonates with you, visit [GitLab's Career Opportunities](#).

Assessing the remote workplace experience

1. While Glassdoor does not yet offer a dedicated Remote Experience filter in reviews, you are able to sort by COVID-19 Related. This filter increases the chances of reading reviews relevant to one's remote work experience at an organization. It's also helpful to search for remote within an organization's Review section. Glassdoor also offers remote-related content on its blog.

{{% include "includes/isthisadviceanygoodremote.md" %}}

{{% include "includes/courseragitlabremotecourse.md" %}}

Contribute your lessons

GitLab believes that all-remote is the future of work, and remote companies have a shared responsibility to show the way for other organizations who are embracing it. If you or your company has an experience that would benefit the greater world, consider creating a merge request and adding a contribution to this page.

[Return to the main all-remote page.](#)

SECTION: company/culture/all-remote/asynchronous.md

How to get started with async

{style="max-width: 50%"}{}

GitLab believes that all-remote is the future of work. The first step to embracing remote work includes understanding asynchronous (async) workflows and communication and how to make async work for you.

What is asynchronous work?

We love Preston W.'s explanation from the Remote blog

> "Asynchronous work is a simple concept: Do as much as you can with what you have, document everything, transfer ownership of the project to the next person, then start working on something else."

Asynchronous work is growing in popularity because it significantly benefits both employees and employers.

Six major benefits of asynchronous working

1. Asynchronous work provides autonomy, empowerment, and agency

In an asynchronous company, team members are given agency to move projects forward on a schedule that suits them. At GitLab, we measure impact not activity. This means that people are free to achieve results when it best suits them.

If someone is traveling to a new time zone each month, or chooses to spend a beautiful afternoon with family, that's their prerogative.

Unsurprisingly, providing those who are capable of being managers of one with this type of autonomy leads to extraordinary loyalty, retention, and quality of work.

To further optimize this approach, consider adding a "flexible time off policy", which means team members do not need to ask permission to step away from work.

All-remote settings empower team members to live and work where they're most fulfilled. Increasingly, operating asynchronously is necessary even in colocated companies which have team members on various floors or offices, especially when multiple time zones are involved.

In this GitLab Unfiltered video, Emna G., founder and CEO at Veamly, speaks with GitLab's Darren M. about the impact of defaulting to asynchronous as it applies to stress, anxiety, mental health, and overall well-being.

{{< youtube "cy6WGuzArgY?start=2019" >}}

Working asynchronously is more efficient, less stressful, and easier to scale.

2. Asynchronous work increases efficiency and boosts productivity

Working asynchronously is highly efficient because everyone on the team can always default to action .

>"There are many times when work isn't ready for us to tackle, tasks aren't planned, decision makers aren't online, etc. In these times, successful teams execute, even if they later have to refactor and adapt, they don't waste time "waiting".

3. Asynchronous work is more inclusive

One of the biggest benefits of asynchronous work is that it completely removes the hurdle of time zones.

>"At GitLab, we have people spread across 60+ countries, so almost every time zone is covered. But, that also means that someone on your team is likely in a vastly different time zone. In fact, they may be asleep the entire time you're up working." - Darren M., GitLab's Head of Remote

Business happens around the clock, in all time zones, in perpetuity. Attempting to shoehorn communications into a single time zone's predefined set of hours is dysfunctional.

As a 100% remote company with team members in more than 65 countries, working asynchronously removes time zone barriers and is another way GitLab is a more globally inclusive organization.

4. Asynchronous work alleviates stress and supports mental health

A tremendous amount of stress comes with expectations to be online, available, and responsive during set working hours. Worse, our hyper-connected society has allowed this notion to seep into every hour of the day, destroying boundaries between work and self.

An unsung benefit to working asynchronously is a reduction of tension. When your entire company operates with an understanding that any team member could be offline at any time, for any reason, there is no expectation that one will reply instantly to an inquiry.

This creates an environment where your mental health is prioritized, allowing team members to set boundaries and freeing them from a perpetual assault of notifications and judgment.

When asked how asynchronous communication impacts a societal expectation for an immediate response at all hours of the day, Darren M., GitLab's Head of Remote, offered the following during an interview with Stuart Miniman, host of theCUBE and GM of Content at SiliconANGLE Media.

> Remote is much better for your mental health and sanity than other settings because it forces you to work asynchronously.

>

>An asynchronous mindset enables everyone to take a step back and assume that whatever we're doing is done with no one else online. It removes the burden of an endless string of messages

you must respond to immediately.

>

> From a mental health standpoint, when you have an entire company that embraces async, we all have a little more breathing room to do deep work that requires long periods of uninterrupted time.

>

> Society is nearing a tipping point where people can't tolerate many more messages, emails, or seemingly urgent pings while trying to do their job well. We may be a bit ahead of the curve on that, but I hope that the industry at large embraces asynchronous communication and allows people more time to do the work they were hired to do.

5. Asynchronous work encourages thoughtfulness and intentionality

A core problem with synchronous communication is the perception of deadlines. When there is an arbitrary start time and end time to a working day, there is pressure to communicate as much as possible between those times, often at the expense of processing time.

Sahil Lavingia, founder/CEO at Gumroad, shares powerful benefits his company realized in going fully asynchronous.

From their X (Twitter) thread:

> All communication is thoughtful. Because nothing is urgent (unless the site is down), comments are made after mindful processing and never in real-time. There's no drama.

>

> Because everyone is always effectively "blocked," everyone plans ahead. It also means anyone can disappear for an hour, a day, or a week and not feel like they are holding the company back. Even me!

>

> People build their work around their life, not the other way around. This is especially great for new parents, but everyone benefits from being able to structure their days to maximize their happiness and productivity.

>

> This is possible because everything is documented. And because everyone talks through different text-based mediums, it's easy for people to peer into anything if they're curious (or take over if need be). There are also no meetings, and all numbers are public, so there's no workplace FOMO.

>

> The software we ship is well-tested and incredibly stable. It has to be, because we're never online at the same time to "deploy" together. There are rarely fires to fight, and we lower the amount of technical debt we have at Gumroad every week too!

>

> Overall, it's a very low stress environment. Many of us don't even have Slack installed. Yet, we're shipping the best software we've ever shipped, and growing faster than ever. Funny how that works!

6. Asynchronous work bridges the knowledge gap

Asynchronous companies should implement a low-context culture. This means that communication is precise and direct. Team members forecast what questions may be asked about a communicate and

add in as much context as possible in its delivery. By assuming that the recipient is asleep, or perhaps doesn't even work at the company yet, this added context removes ambiguity and decreases the likelihood of misinterpretation.

This may feel inefficient, as crafting a message may take longer to compose and edit. However, the long-term benefits are remarkable. At GitLab, we have years of documented decisions · such as this example of availability over velocity · loaded with context. This enables new hires to self-learn as they sift through archives to discover the context that influenced a given decision.

Synchronous organizations often make decisions in a series of meetings, documenting little to nothing along the way, such that those who come into the process mid-stream are constantly wasting cycles on fact-finding missions. Plus, those hired after a significant decision is made cannot understand the context of something that was changed before their arrival, creating cavernous knowledge gaps that eat away at a company's efficiency.

Coda Hale, principal engineer at MailChimp, articulates this well in a comprehensive article on organizational design entitled *Work is Work*.

- > A significant source of failure demand for meetings and status updates is the desire of organizational leaders to keep abreast of who's doing what. This situational awareness is indeed important, but trying to maintain it by calling meetings, messaging people on Slack, and catching people on the hallways is a significant systemic drag on organizational productivity.

- >

- > A better model for staying informed of developments as the organization scales is for groups to publish status updates as part of the regular cadence of their work. Leaders can asynchronously read these updates and, should the need arise, initiate additional, synchronous conversation to ask questions, provide feedback, etc.

- >

- > Synchronous meetings should be reserved for low-latency collaboration on complex issues; likewise, collaboration should be reserved for synchronous meetings. · Coda Hale

As companies scale, people will come and go. By utilizing asynchronous communication, an organization is able to retain knowledge throughout these natural cycles.

For example, the Git blame history of GitLab's Values page shows a complete list of who made what change, and what the context was for each of them. This insight is invaluable, as some contributors no longer work at GitLab. Those seeking information can find it asynchronously without bothering anyone else, nor do they have to wait for anyone else to wake up or come online.

Guide to asynchronous communication

Schedules and calendars have conditioned us to operate in synchronicity · when two or more parties are in the same place (either physically or virtually) at the same time.

However, we now live in a world where asynchronous (async) communication allows us to move projects forward without requiring stakeholders to be synchronously physically or virtually present. Async communication optimizes how (and when) people work and communicate.

How does asynchronous communication work?

Fundamentally, asynchronous communication is simple. We do it all the time, when we send messages, leave voicemails, and record videos. Communicating async just means that the recipient of the message and the sender are unlikely to be in the same space at the same time.

However, doing async communication well requires significant intentionality. When creating an async message, you have to consider questions like:

- Am I using the best format (written, verbal, video...) for this message?
- Does the person receiving this message have all the context they need?
- Am I communicating clearly so that there will be no confusion?
- Am I considering the tone of my language and how this message will be received?
- Am I providing any needed resources or next steps, so that the conversation or project can move forward?
- Is this communication being documented in a way that it can be found later for future reference?

This level of thoughtfulness often produces communications that are clear, complete, delivered with kindness, and that create productive results.

This also means that communicating asynchronously takes more time and planning, and requires specific tools. When it comes to async communication, there is as much to unlearn as there is to learn.

The investment of time and strategy is worth it: communicating well asynchronously creates major improvements in efficiency, and supports strong collaboration and teamwork.

You can't do async communication without strong documentation

The importance of strong documentation for async communication truly can't be overstated. No matter how intentional you are in communication, there's always something that will be left out, misunderstood, or needed to move forward. If someone has a follow-up question, they may need to wait hours or days for a response. Alternatively, they can look up the answer in the company handbook.

GitLab has a handbook-first approach to all communication. Our goal is to ensure that our handbook is always up to date and that it is a powerful resource to make our team massively more efficient. The GitLab handbook would be over 2,000 pages if printed, and it is available to read for any visitor who wants to know how we work.

While you may not choose to have this level of transparency, be aware that transparent information-sharing within your organization is crucial to asynchronous work. Every team member should be empowered to do their work at any time, whether or not their teammates are online and available.

Using GitLab for remote collaboration

GitLab's entire team uses GitLab to collaborate asynchronously on all of our work. GitLab is a collaboration tool designed to help people work better together whether they are in the same

location or spread across multiple time zones.

Originally, GitLab let software developers collaborate on writing code and packaging it up into software applications. Today, GitLab has a wide range of capabilities used by people around the globe in all kinds of companies and roles.

You can learn more at [GitLab's remote team solutions page](#).

When to use asynchronous instead of synchronous communication

While GitLab has a bias towards asynchronous communication, a strategic balance between synchronous and asynchronous is useful for achieving maximum efficiency. Working asynchronously is not a goal unto itself; rather, being considerate and opting to move a discussion or project forward asynchronously when feasible creates more space for synchronous moments.

Highly capable asynchronous work still allows for, and includes at appropriate moments, some synchronous discussion. Async is very powerful for GitLab, but not an absolute · especially if at the expense of our values.

Why verbalize and write when you could just write?

At GitLab, if you schedule a work-related meeting (e.g. not a coffee chat) it is required that you have an agenda. If you add an agenda item, you are expected to verbalize your agenda item and ensure that you or someone else is taking notes of the response. If writing it down effectively communicates the intent, then consider going completely asynchronous on the topic.

If you are creating double work for yourself or others · holding a meeting simply to document what will need to be written down in order to work handbook-first · it is likely more efficient to not hold a meeting and instead work asynchronously.

When considering meetings, review the GitLab value of Efficiency and following the meeting guidelines in being respectful of others' time. Do not schedule a coffee chat which is a work-related meeting in disguise.

How to implement asynchronous workflows

Ask, "Do different conversations require different modes of communication?"

The easiest way to take on an asynchronous mindset is to ask this question: "How would I deliver this message, present this work, or move this project forward right now if no one else on my team (or in my company) were awake?"

This removes the temptation to take shortcuts, or to call a meeting to simply gather input. (After all, every meeting should be a review of a concrete proposal, and only called when it will lead to a more efficient outcome than would be possible asynchronously.)

Focus on iteration

Practicing iteration · at GitLab requires intention and effort. It is often referred to as the most difficult value to embrace. Iterating on numerous ongoing projects is an ideal forcing

function to ensure a bias toward asynchronous communication.

Asynchronous work can feel taxing and inefficient if you're only working on a single project and you're stuck waiting for another person's contribution. Scheduling your work so you can pick up other tasks while waiting to be unblocked can reduce this downtime.

Now suppose you have five ongoing projects. It's much easier to make iterative progress on one, tag a person or team within a GitLab epic, issue, or merge request for desired input or action, and switch to another ongoing project while you wait. Suppose you cycle through your assigned tasks, making iterative improvements on each before handing off. In that case, you're able to create minimal valuable change for many more projects while being less concerned over the immediate response to any one of the projects in particular.

Async works well when managing multiple projects, although it does require discipline and an ability to switch contexts and compartmentalize.

{{< youtube "okcPC9YucA" >}}

GitLab co-founder Sid and the Learning and Development team provide more context on our bias towards asynchronous communication and the importance of our iteration value.

> There is a reason we are really good at async, and that is because we make things smaller. Through iteration, you don't have to coordinate with a ton of people. By taking smaller steps through iteration, we can ship faster. The only way this is possible is through asynchronous communication. · Sid Sijbrandij, GitLab co-founder

Aim for progress, not perfection

At GitLab, everything is in draft mode and subject to change. Asynchronous workflows are more easily adopted when you foster a culture of progress over perfection. Move a project forward as best you can given the resources available, and if you reach a point where you're blocked, attempt to ship what you have now through a two-way door.

This allows colleagues to see the direction you're heading and relieves pressure on them to reply immediately as some progress is better than none.

Celebrate incremental improvements

Asynchronous workflows require a culture where incremental improvements are celebrated equally, if not more, than massive launches. If leadership casts shame on unfinished or unpolished work, workers will be reluctant to work asynchronously. Rather, they will delay work for the sake of consensus. Consensus feels good, but can easily mask inefficiency, progress, and innovation.

Document accurately

Mastering the art of communicating asynchronously has a prerequisite: documentation. At its core, asynchronous communication is documentation. It is delivering a message or series of messages in a way that does not require the recipient(s) to be available · or even awake · at the same time.

If your organization has no standardized method of documentation, establish that first.

Use the right tools

Asynchronous communication works best when there is company-wide alignment on how and where to input communication. Leaders should carefully select their tools, aiming to direct communications to as few channels as possible.

A common frustration in large organizations - regardless of what stage of remote they're in - is the chaotic splintering of communication. Projects frequently end up strewn across email, chat, text messages, unrecorded meetings, design tools, Google Docs, and more. It's best to choose a single system for communicating project progress.

At GitLab (the company), that destination is GitLab (the product). Any side conversation that occurs in a meeting is documented in an agenda, and the useful elements are contextualized and ported to relevant GitLab issues and/or merge requests. The same goes for side conversations that happen in Slack or email. Relevant portions are ported over into GitLab (the product), which is the single source of truth for any ongoing work.

If it's not in a GitLab epic, issue, or merge request, it doesn't exist. This mentality is essential to reaping the benefits of asynchronous communication.

Here are some tried-and-true apps and cloud tools that facilitate asynchronous collaboration:

- Google Drive
- Dropbox Spaces
- Qatalog
- Yac
- Friday HQ
- GitLab
- Tettra
- Slab
- Twist

Minimize time zone bias

Leaders should strive to eliminate bias toward one time zone, or one swath of time zones (e.g. time zones covering North America). For company all-hands meetings, look to rotate these to accommodate a more diverse array of time zones. Also consider recording them so that others can watch at a later time.

When hosting live learning sessions, for instance, host several instances so people around the globe are able to attend one that suits their schedule.

If a company pulls too hard in the direction of one time zone (oftentimes the zone where most company executives live), it signals to the rest of the company that asynchronous workflows aren't taken seriously.

Best practices for asynchronous workers

{style="max-width: 50%"}

Create mental space

One of the more challenging aspects of remote work is closing out of all mental tabs · to use a web browser analogy · once you leave work. Since remote enables you to work a non-linear workday, it's difficult to rationalize where one working session ends and another begins. There is often no reason or excuse other than "it's time." Dedicated remote workers may struggle with disconnecting from this feeling, deprioritizing their own well-being and falling into the trap of "just one more reply."

Chat-style communication tools should be used primarily for informal communication. If you are accustomed to Slack being an always-on center of urgency in a prior organization, breaking your reliance on it as a core part of accomplishing tasks will require deliberate effort and reinforcement.

Below are recommended forcing functions to keep leaders and individual contributors alike from being consumed by instant messages and a bias for synchronicity. The goal is to place the power of prioritization back into one's own hands. This is critical to being an effective manager of one.

Clear all messages daily/weekly

Humans were not designed to receive an unchecked quantity of new data in perpetuity. For many, it would be a full-time job to simply read and comprehend a daily or weekly digest of new Slack messages, private and public. While an individual's approach to filtering what is vital and is not will differ by role and function, you can reduce your mental load by clearing all messages at the end of each working day or week.

Slack refers to this as Mark all messages as read, which is easily toggled by simultaneously pressing Shift + Esc.

Communicate your work preferences

Create a rudimentary README that clarifies how you work. Ideally, it's working from a GitLab Issue board, tagging system, or To-Do list which can be understood and used company-wide. You can then post the link to your README in your Slack profile, pointing others to it. Showing others how to deliberately choose asynchronous over synchronous is vital to reinforcing our operating principle of Bias towards asynchronous communication.

This is an extension of another remote-first forcing function: Always answer with a link.

Communicating your async work days

GitLab embraces asynchronous work and non-linear workdays broadly, but there may be times when you're working outside your normal work hours (e.g. while traveling before/after a trip, working during a trip, working a weekend, etc.) and want to communicate that to team members. In these situations, team members should add a Focus Time calendar event and change their Slack status rather than entering time off in Workday.

Remove Slack from your phone

Remote workers lack many of the physical gateways that serve as dividers between work and life. When work and life happen in the same building, and one's work equipment is always within reach, it's far too easy to allow unread Slack messages to haunt you.

Being intentional about removing Slack from one's phone is a great way to reinforce that time away from work is important. A litany of studies have covered the addictive impact of smartphones. Even if you aren't sure if this approach will benefit you, give it a try. It's a two-way door.

Be transparent on capacity

In a colocated setting, a worker can pick up context clues by seeing someone storm away, sigh loudly, or intentionally put on a pair of noise-cancelling headphones to prevent interruptions. In a remote setting, it's not as simple to let people know you need uninterrupted focus.

Thus, it's important to leverage Slack statuses to broadcast information on your capacity to your team. You should feel safe to manually adjust your status to indicate when you are at capacity or engaged in focus time. This reinforces that others can and should consider doing likewise, while also reminding others that Slack and synchronous conversation should not be the default.

Remind people that async is more inclusive

While GitLab's approach to self-service and self-learning is reinforced during onboarding, continual reinforcement may be necessary. It is acceptable to ask someone if they are exercising a bias towards asynchronous communication, regardless of their position on the org chart.

Just as we would hope that all GitLab team members would be quick to ask if something is inclusive, it's important to remember that asynchronous communications is another way for GitLab to be more inclusive.

There is as much to unlearn as there is to learn.

Use accurate terms and identifiers

It is important to use accurate names when referring to people. To do so, use the @ symbol in GitLab Issues/Merge requests, Google documents, and Slack to clarify to whom you are referring. This avoids accidentally misspelling someone's name or referring to the wrong person.

Examples:

- Names that may be difficult to spell. E.g.: Hubert Wolfeschlegelsteinhausenbergerdorff
- Names that could refer to multiple people. E.g.: There are multiple GitLab team members with the name Tatiana.

How to manage asynchronous meetings

{style="max-width: 50%;"}

At GitLab, we have a bias towards asynchronous communication. As a meeting participant, whether you are scheduling or an invitee, question every work-related meeting.

1. What is the outcome I am trying to achieve that has led to my desire to schedule a meeting?
1. Can the desired outcome be broken down into smaller tasks?
1. Can the desired outcome be achieved or worked towards by dogfooding and using a GitLab issue or merge request?
1. Am I trying to gather consensus? (If so, this can be done asynchronously.)
1. Am I trying to make a decision after consensus is gathered and there is a proposal to react to? (If so, a meeting may be acceptable if it cannot be agreed upon asynchronously, but remember that outcomes must still be documented in the handbook. If your outcome(s) will be documented in the end, it calls into question the efficiency of a synchronous meeting.)

When to start synchronously

When it's clear that a kickoff meeting is useful to build rapport, trust, and quickly deliver shared context to a group, consider starting a project synchronously. This initial engagement should be used to establish a working foundation, such that future touch points can be primarily asynchronous.

Starting with a synchronous kickoff may make sense for the following. (Thanks to the team at Dropbox for articulating this well in its remote communications guide.)

1. A sales engagement
1. First-time meetings with external parties
1. First-time meetings with GitLab team members who have not previously worked together
1. One-way door decisions (e.g. when stakes are high and decisions are difficult to reverse)
1. Complex initializations (e.g. defining a corporate narrative, a major overhaul to scope, etc.)
1. Emotionally sensitive topics (e.g. discussing personal issues, career path/promotion, difficult feedback, etc.)
1. Supporting and unblocking your direct reports (e.g. a regular 1:1)
1. Celebrations and retrospectives (it feels good to celebrate wins with a group, and lightweight retrospectives can serve as kickoff points for future sprints)

When to start asynchronously

Suggesting to "hop on a quick videocall" may feel insignificant, but it can have a negative impact on productivity. Generally, it's best to avoid a meeting for the following items.

1. Status updates
1. FYIs and process documentation
1. Meeting about a meeting

For an example of starting async first in a highly complex scenario: View Vision Statement: gitlab.tv (Vision Statement: gitlab.tv). In this scenario, the project began asynchronously, as the initiator needed to convey a complex series of assumptions and examples in order to provide

enough context for team members to contribute feedback.

1. Due to the large volume of information, the creator of the Epic added a series of Highlights at the top, an indication of being respectful of others' time.

1. The creator time-boxed feedback, providing a specific window of time that asynchronous input would be accepted.

1. The creator transparently notified the group that a focused synchronous meeting would follow in order to make a decision on the asynchronous feedback and proposals.

When to pivot from asynchronous to synchronous

When a back-and-forth asynchronous conversation is moving very slowly with a high volume of small statements between two people, sometimes a quick synchronous discussion leads to a quick micro-resolution. Generally, if two people go back-and-forth more than three times on the exact same topic · and it's impractical to break it into smaller async-friendly decisions · it makes sense to temporarily pivot to synchronous or leverage a richer communication medium such as Yac or Loom.

These tools allow messages to be conveyed asynchronously, though the use of audio and video as the medium may enable deeper connections to be made compared to raw text transmissions.

Following pivots to synchronous calls, there should be a written summary created to inform others of the outcome, ideally shared in a relevant GitLab epic, issue, or merge request.

Best practices for asynchronous meetings

{{< youtube "amiDcpIXIQ8?start=301" >}}

In this GitLab Unfiltered video, two GitLab colleagues discuss lessons learned from managing a team asynchronously, across many time zones, while traveling.

Focus on the positives

Working asynchronously allows teams to have fewer meetings. Initially, the notion of "optional meetings" may seem absurd to those who are accustomed to synchronous communication. The truth is that you're either at a meeting to contribute, or you aren't.

The beauty of asynchronous is that team members can contribute to meetings that occur while they sleep.

Meeting attendance becomes optional when each team member has access to an agenda and a Google Doc, which should be attached to each invite. This allows people anywhere in the world to contribute questions/input asynchronously in advance, and catch up on documented outcomes at a later time.

The person who called the meeting is responsible for contextualizing the outcomes and porting relevant snippets to relevant GitLab issues and/or merge requests.

The organizer is responsible for informing the entire company, via post-meeting documentation, of the outcomes should team members go searching. That's a big responsibility, which keeps the

quantity of meetings in check and acts as a filter for whether or not a meeting is truly necessary.

Set KPI targets to track improvements due to asynchronous work

Team needs are as unique as their objectives and will experience asynchronous communication differently. However, team leaders and managers can set KPI targets based on one or all of the following:

- 1. Percent reduction of minutes spent in synchronous meetings
- 1. Percent increase in GitLab issue/epic/merge request usage
- 1. Improved ratio of meeting minutes-to-GitLab contributions
- 1. Increase in 360 feedback citing 'bias towards asynchronous communication' in the Diversity, Inclusion & Belonging field

How to have an async 1:1 meeting

Async 1:1 meetings are an excellent way to broaden the communication skills of a manager and a direct report. While face-to-face or walk-and-talk 1:1s are beneficial, the ability to cover 1:1 agenda items asynchronously using written text bolsters one's overall remote competency.

To conduct an async 1:1, be sure to

- 1. Communicate to the other party that you would prefer the next 1:1 to be async to ensure mutual understanding of the format.
- 1. During the week of the async 1:1, consider the entire working week fair game to make comments in an ongoing agenda document. Tag the other party with FYI or FYA commands and provide written or embedded video context, understanding that their feedback or input will not happen in real-time.
- 1. Consider leaving the original 1:1 calendar block on your schedule. If you are not able to look at the agenda document before that block, this ensures dedicated focus time to respond to agenda items or add new ones.
- 1. For any agenda items which could not be addressed during the week of the async 1:1, move those to the following instance and resume face-to-face or walk-and-talk.

Retrospectives on meetings

For existing and upcoming meetings, add this question at the top or bottom of the agenda and document the answer: Could this meeting have been handled asynchronously, and if so, how?

Consider sharing these learnings in a public channel to create additional awareness of what's possible through asynchronous workflows. Take time to reflect on which meetings you've attended or scheduled in recent weeks. Which were a valuable use of time and which could have been handled asynchronously?

There are many tasks which can be handled synchronously and asynchronously. The goal is to always select asynchronous where feasible, creating more focus time in your day. This also creates a higher likelihood that team members will have more energy for synchronous connections that bolster work relationships. Informal communication is vital in an all-remote setting; by being ruthless in one's bias towards asynchronous work, it creates more space for synchronous

team bonding. We each have a finite amount of tolerance for work-related meetings and video calls; synchronous moments are better saved for informal communication such as coffee chats and team trivia where feasible.

Limitations and challenges of asynchronous communications

Asynchronous communication has its limits. Although projects are moved forward asynchronously at GitLab, there are times when portions of the project are best handled synchronously.

Evaluating efficiency

As a rule, when team members at GitLab go back and forth three times, we look to jump on a synchronous video call (and document outcomes).

Client-facing roles

Certain roles are more tolerant of asynchronous communication than others. Client-facing roles, for instance, may have certain requirements for coverage during certain hours. It's possible to layer async work atop these demands by ensuring that there is no single point of failure, such that a team within an asynchronous organization can self-organize and decide who covers given time slots.

Time zones

While communicating asynchronously is an excellent way to reduce the pain of having team members spread across an array of time zones, managing this as a small team is particularly challenging. For example, a small team which is primarily based in North America may struggle to communicate well with the first team member who joins from Singapore given the time zone difference.

However, as a team scales and more coverage is added in time zones in between, it's easier to hand work off as the world turns. In many ways, managing time zones becomes easier with scale, as the delta between teams is reduced.

Interviewing external candidates

All of GitLab's interview processes involve some form of synchronous communication. Some of our teams utilize asynchronous practices during the interview process, however, this is not a standard approach across every interview process.

Working async outside of your company

Even if you have established asynchronous work within your team, it can be challenging or uncomfortable to encourage async practices when working with people outside of your company.

Every organization has their own norms, but you can politely challenge the status quo by seeking to inform and educate. Here's how to approach working async with external parties:

1. Start synchronously: Have a sync call to kick off a project or partnership. During that call, mention that you'd like to incorporate some asynchronous work into the project moving

forward. If you use this call to build relationships and rapport, your async work will be more efficient.

1. Set expectations and model behavior: Discuss up front how you will work async together (what tools you'll use, what to document, and how often). Agree on a cadence for any future syncs so that everyone knows what to expect. Be sure that you model this behavior throughout the project or partnership.

1. Share async documentation: Send your handbook page or documentation about async to people you're working with externally. This way, anyone unfamiliar with working this way can refer to the page and share it with others.

Some organizations may not be open to working asynchronously, so it's important to remain flexible, particularly in a client-facing role. However, imagine how much time could be saved if more companies had a bias for async?

Async at GitLab

{style="max-width: 50%;"}

Asynchronous communication is a significant differentiator in a world where businesses are increasingly remote. GitLab's seeks to more clearly define and operationalize asynchronous communication.

Over time, GitLab, the company, has iterated on its asynchronous culture:

1. Iteration 1: GitLab operated largely async through shared observed behaviors as a small team
1. Iteration 2: we began to build a playbook and define when synchronous and asynchronous was appropriate
1. Iteration 3: we have defined best practices and intentional operations.

We believe this will:

1. Help GitLab maintain its leadership in remote work.
1. Reduce ambiguity in communication modes and reinforce handbook-first.
1. Allow for added flexibility in the day-to-day lives of GitLab team members.
1. Support our long-term vision for global remote work.

Examples of asynchronous integration on GitLab teams

Activity	Async Communication
-----	-----
Weekly announcements	Engineering Management creates a weekly announcement video and slides to be viewed async, at a time convenient for each team member.
New team member introduction	New team member creates a 2-minute video introduction, introducing themselves to the team which can be shared in meetings / Slack channels
Backlog refinement / planning poker	Team collaborates via a GitLab

issue (or Epic, or merge request, if more relevant), tagging the appropriate parties with specific requests. If the volume of information is above 1,000 words, ensure that a Highlights section is at the top. |

| Capacity planning monthly. | Team updates a shared Google Sheet

| Team members who are unable to attend sync meetings | Meeting organizers should affix a Google Doc agenda to each meeting invite prior to sending. Team members invited to the meeting should update the meeting agenda and questions async, or pre-record a video with information to be shared (linking the video in the agenda). |

| Quarterly team results recaps and celebrations | Corporate Marketing (corp-mktg) creates a Google Doc or Slides for team members to add results async, then shares the resulting celebration video (with those could attend sync) on GitLab Unfiltered. |

| Monthly finance accruals | DRIs (Directly Responsible Individuals) in respective departments set a monthly personal reminder to update ongoing Google Sheets with the latest financial accruals, tagging their finance partner(s) in the doc if there